

MediWatch

Informe Tecnico de la Solucion

Plataforma de IA para monitorear disponibilidad, riesgo de desabastecimiento y precios de medicamentos en Colombia, con datos abiertos oficiales.

Reto de salud - "Datos al Ecosistema 2026: IA para Colombia" (Nivel Avanzado)

Julio de 2026

1. Problema y enfoque

El desabastecimiento de medicamentos vitales afecta a pacientes, IPS y tomadores de decision en Colombia. La informacion oficial existe (INVIMA, MinSalud, CNPMDM) pero esta fragmentada en datasets sin llaves comunes. MediWatch integra 4 fuentes oficiales de datos.gov.co en una plataforma con: (1) un score de riesgo de desabastecimiento interpretable 0-100 validado con backtest temporal, (2) precios de referencia etiquetados por fuente y (3) un asistente conversacional que responde unicamente con datos recuperados y cita sus fuentes.

Hallazgo estructural del EDA

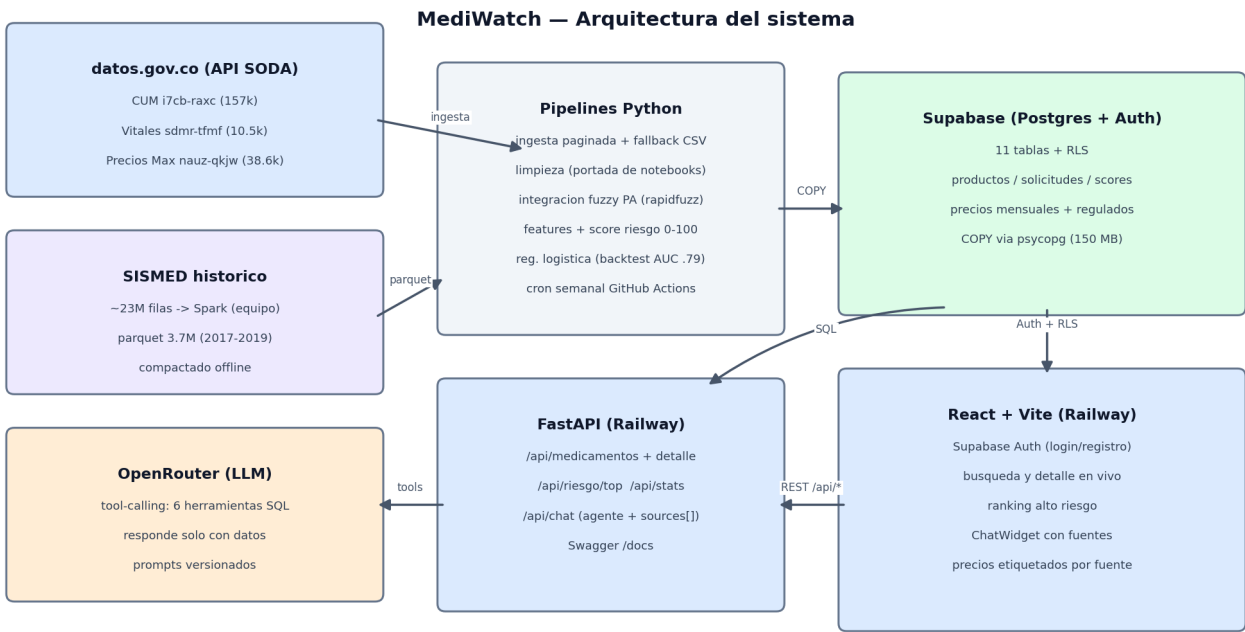
La columna IUM del catalogo CUM esta 100% vacia (la API ni la expone): no existe llave directa entre el catalogo y las senales de escasez. La integracion se resolvio por principio activo normalizado con cascada exacto -> fuzzy (rapidfuzz >= 90) -> componentes de combinaciones, cubriendo 27,5% de los nombres (32,4% de las solicitudes). El analisis del 'sin match' revelo el hallazgo de fondo: gran parte de los medicamentos vitales no disponibles no estan en el CUM vigente precisamente porque carecen de registro sanitario activo - el bajo cruce ES el fenomeno de desabastecimiento.

2. Datos utilizados (4 datasets oficiales)

- CUM Vigentes (INVIMA, i7cb-raxc): ~157.000 filas; catalogo maestro (9.655 productos).
- Medicamentos Vitales No Disponibles (INVIMA, sdmr-tfmf): ~10.500 autorizaciones de importacion excepcional; crece ~940 filas/mes; motor del score de riesgo.
- SISMED historico (MinSalud): ~23 millones de registros compactados con Apache Spark a 3,7M filas (2017-2019); unico dato de precio para el 46% del catalogo; agregado a 724k filas servidas.
- Precios Maximos de Venta (CNPMDM, nauz-qkfw): 38.636 techos regulados vigentes (Circular 19/2024); 30% del catalogo.

Ingesta paginada en vivo via API SODA con fallback a snapshots CSV; re-ingesta semanal automatizada con GitHub Actions.

3. Arquitectura



4 datasets oficiales de datos.gov.co · pipeline reproducible con un comando · trazabilidad de extremo a extremo

Pipeline reproducible con un comando (python -m src.data_pipeline.run_all): ingesta -> limpieza (portada 1:1 de los notebooks de EDA del equipo) -> integracion fuzzy -> features -> carga a Supabase (COPY, 150 MB, RLS). API FastAPI con Swagger. Frontend React con Supabase Auth. Asistente via OpenRouter con tool-calling sobre 6 herramientas SQL.

4. Modelo de riesgo y métricas verificables

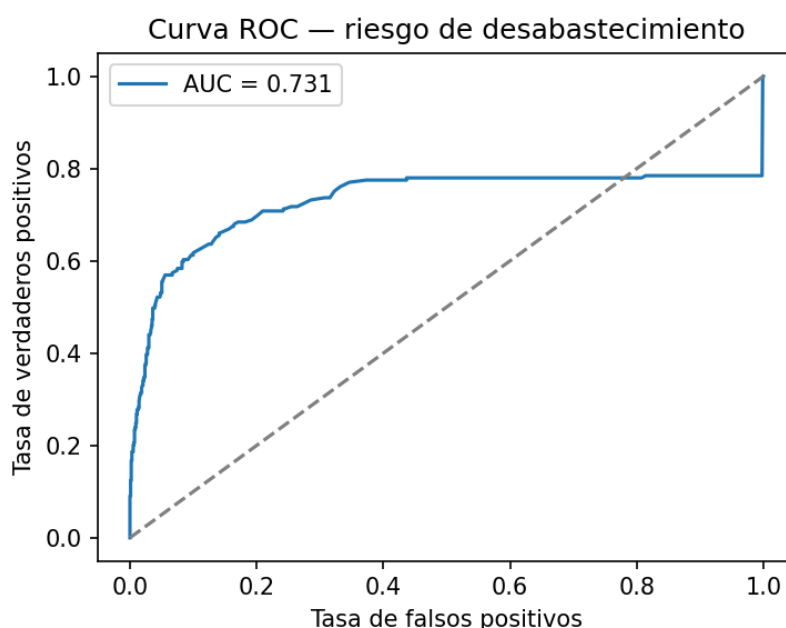
4.1 Score compuesto interpretable (0-100)

Suma ponderada documentada en `config/risk_model_params.yaml`: frecuencia de solicitudes 12m (0,35), tendencia trimestral (0,25), recencia (0,20), amplitud de solicitantes (0,10), urgencia clínica (0,10). Cada score expone sus factores en JSON (trazabilidad) y alimenta una explicación en lenguaje natural generada de forma determinista (sin riesgo de alucinación).

4.2 Validación predictiva (regresión logística, backtest temporal)

Label: 'el principio activo registra solicitudes en los 3 meses siguientes'. Panel PA x mes con features construidas solo con pasado. Backtest en 4 cortes (2024-06 a 2025-11): entrenar hasta el corte, probar los 3 meses posteriores.

- AUC promedio: 0,787 (por corte: 0,76 / 0,80 / 0,86 / 0,73)
- `precision@20` promedio: 0,988 - de los 20 PA más riesgosos según el modelo, ~99% registraron solicitudes en los 3 meses siguientes
- Figuras: matriz de confusión, ROC, distribuciones y correlaciones en `reports/figures/`



4.3 Pruebas de equidad (tests/bias_tests/)

Cinco pruebas automatizadas: invarianza al nombre, no supresión de medicamentos huérfanos, no alarmismo sin evidencia, techo de influencia de la urgencia (10%), y distribución real no degenerada (<50% críticos).

5. Asistente de IA con trazabilidad

Patrón de agente con tool-calling (OpenRouter, gpt-4o-mini): el LLM solo puede responder con datos que recupera de 6 herramientas SQL (búsqueda, riesgo, historial, precios, alternativas, estadísticas). Cada respuesta incluye sources[] con el dataset y número de registros consultados, visible en la interfaz. El system prompt (versionado en models/llm_rag/) prohíbe inventar datos y dar consejo médico. Decisión de diseño: retrieval estructurado SQL en lugar de embeddings - para datos tabulares con cifras exactas es más preciso, más barato y totalmente trazable.

6. Calidad, automatización y despliegue

- 36+ tests (unitarios, integración contra la BD real, equidad) ejecutados en CI en cada push.
- Re-ingesta semanal programada (GitHub Actions) con pipeline idempotente por tabla.
- Despliegue: Railway (2 servicios Docker: API y frontend estático); guía paso a paso en docs/despliegue.md.
- Seguridad: RLS en todas las tablas, secretos solo en variables de entorno, chat_logs privado por usuario.

7. Resultados, limitaciones y próximos pasos

Resultados clave: 9.655 productos monitoreados, 591 principios activos con score (155 en riesgo alto o crítico al corte 2026-04), 628 productos del catálogo enlazados a su score, precios para el 63% del catálogo. Limitaciones declaradas: SISMED es histórico 2017-2019 (siempre etiquetado), la señal de riesgo depende de las importaciones excepcionales, cruce del 27,5% explicado por la naturaleza del fenómeno. Próximos pasos: memoria de largo plazo del asistente (pgvector ya instalado), alertas por correo, diccionario INN/DCI para mejorar el match.